SÃO
PAULO
TECH
SCHOOL

Sistemas Operacionais

Containers

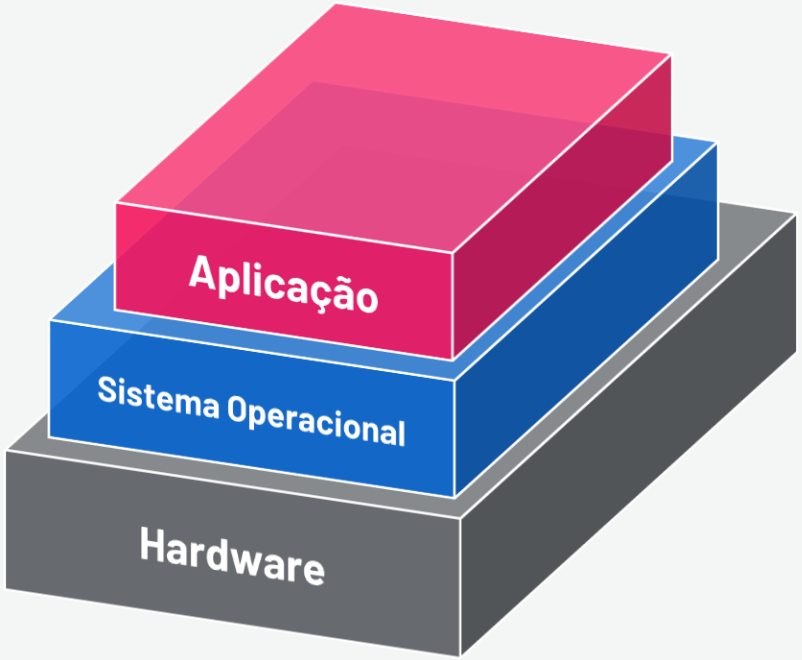
Eduardo Verri

eduardo.verri@sptech.school

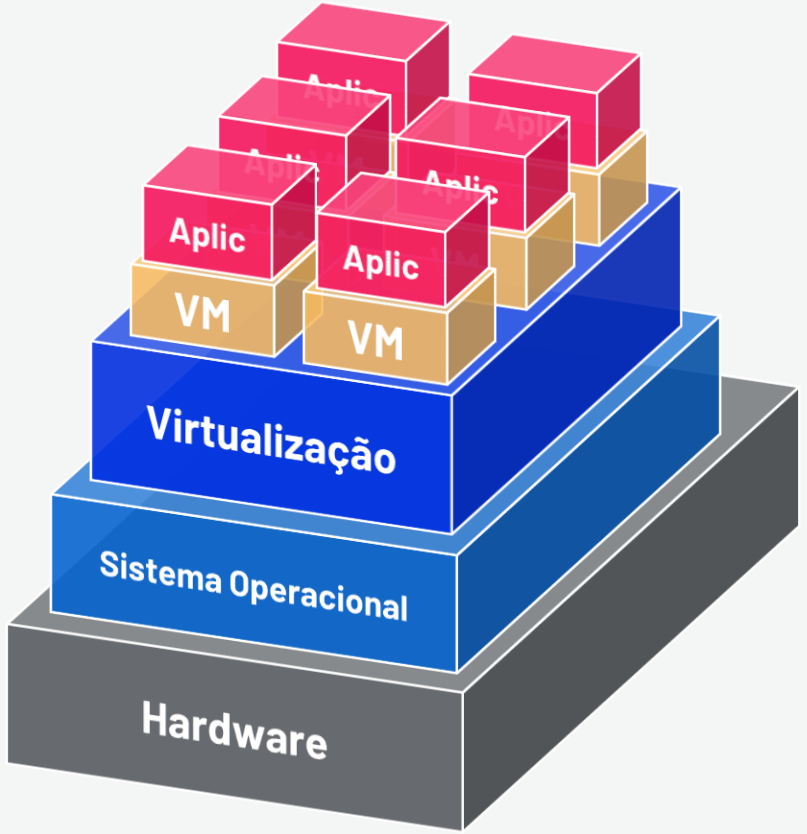
Tópicos da Aula

- O que é Container?
- Arquitetura Virtual Machine vs Container
- Arquitetura Virtual Machine
- Arquitetura Container
- Orquestração de Container
- Observações do Container

Modelos



Tradicional

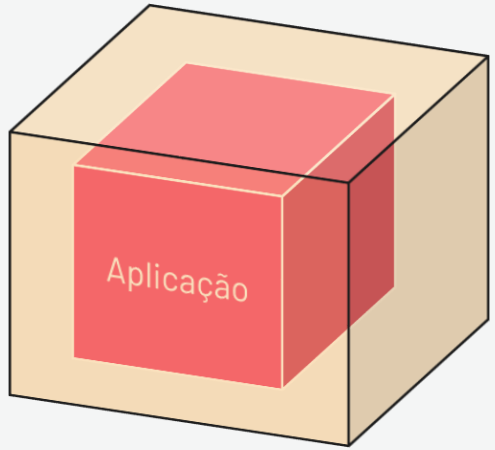


Virtualização



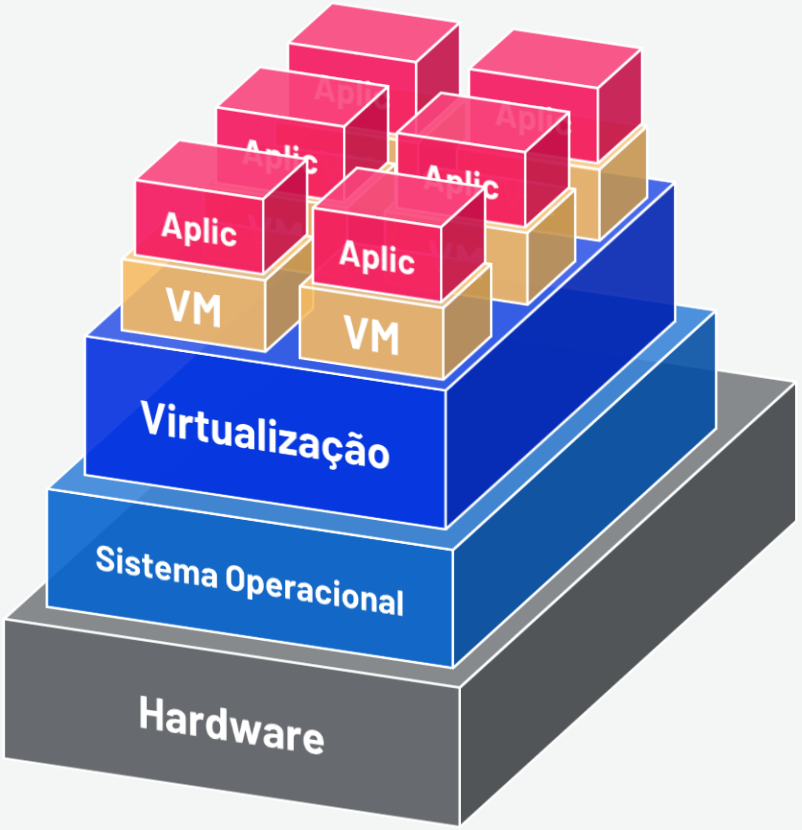
Container

São uma abstração na camada do software que **empacota** código e dependências juntos.



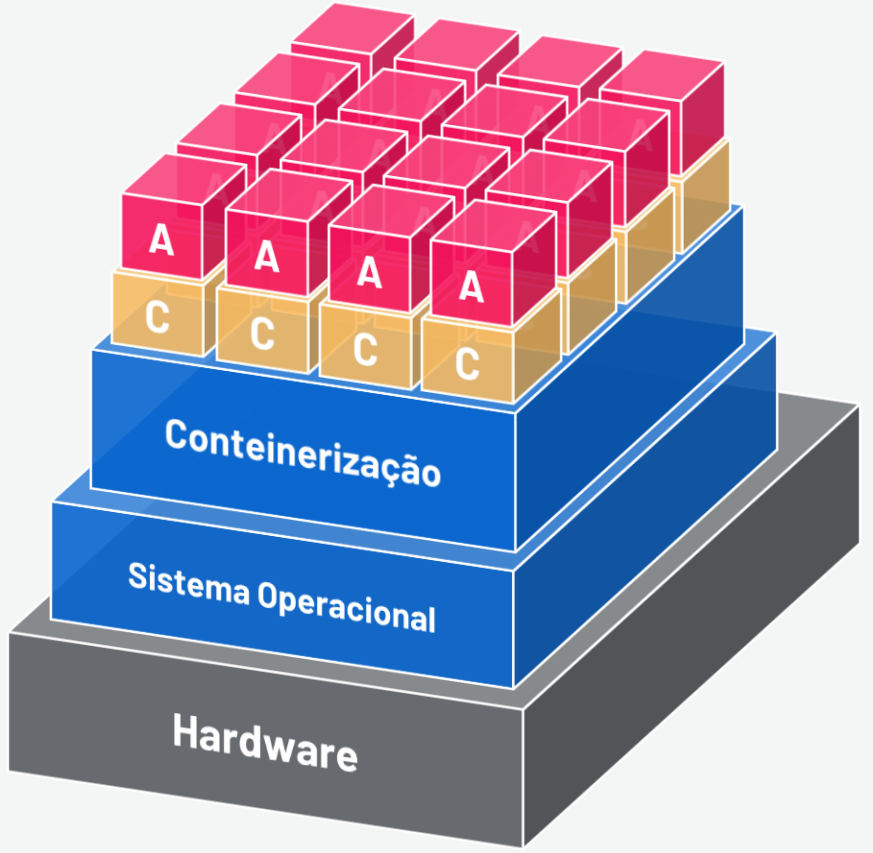
CONTAINER

Modelos



Virtual Machine

+ Consumo de Recursos
+ Tempo de Criação [Pesado]

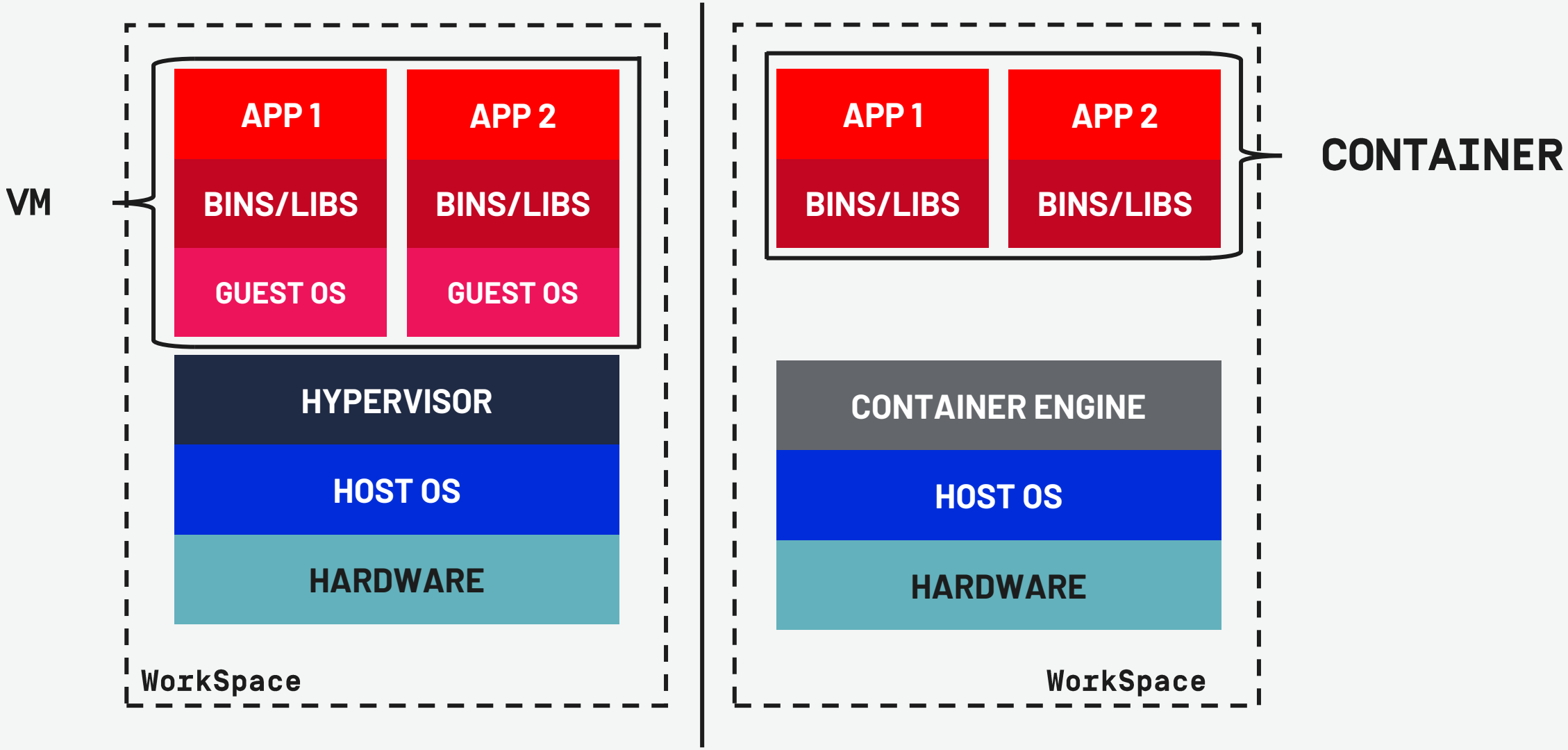


Containers

- Consumo de Recursos
- Tempo de Criação [Leve]

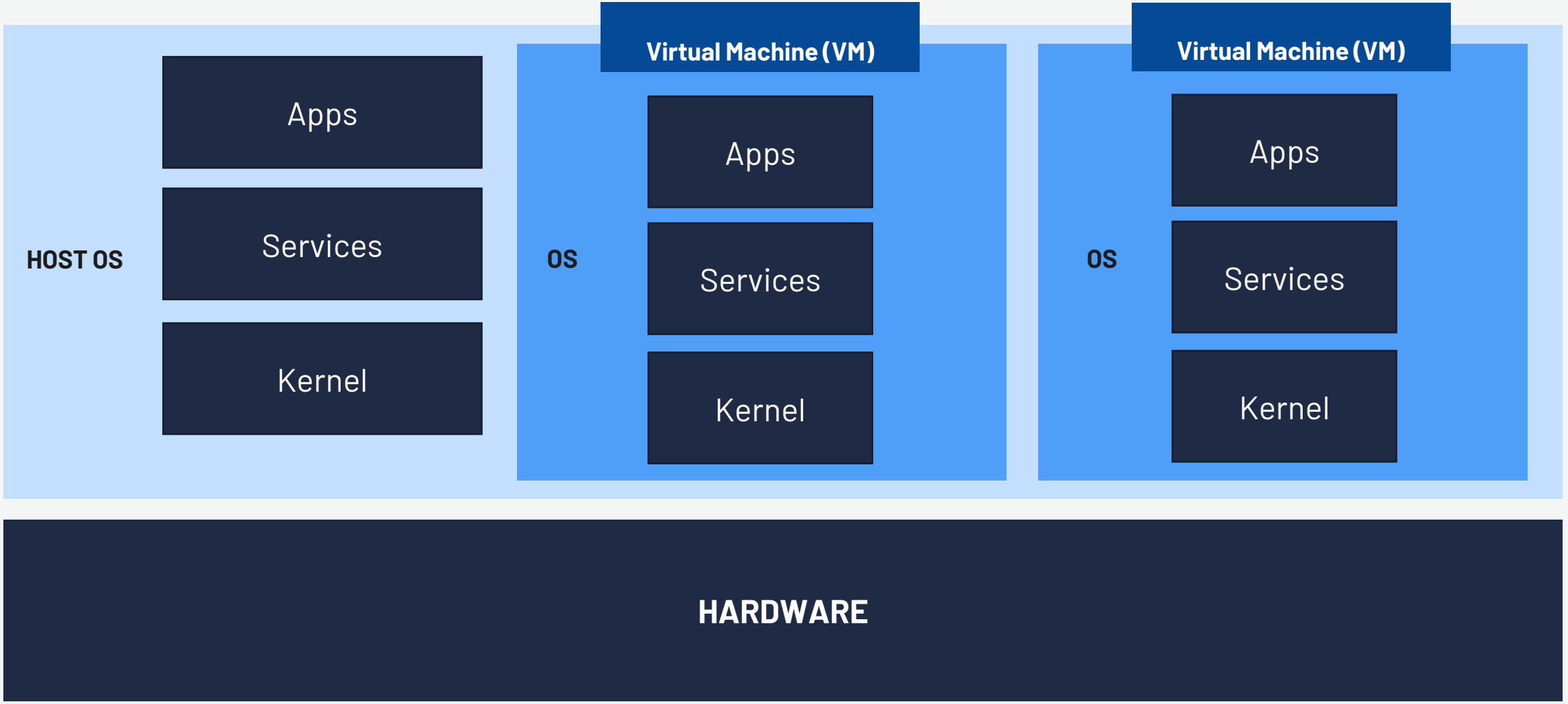
Virtual Machine[VM] vs Container

Arquiteturas



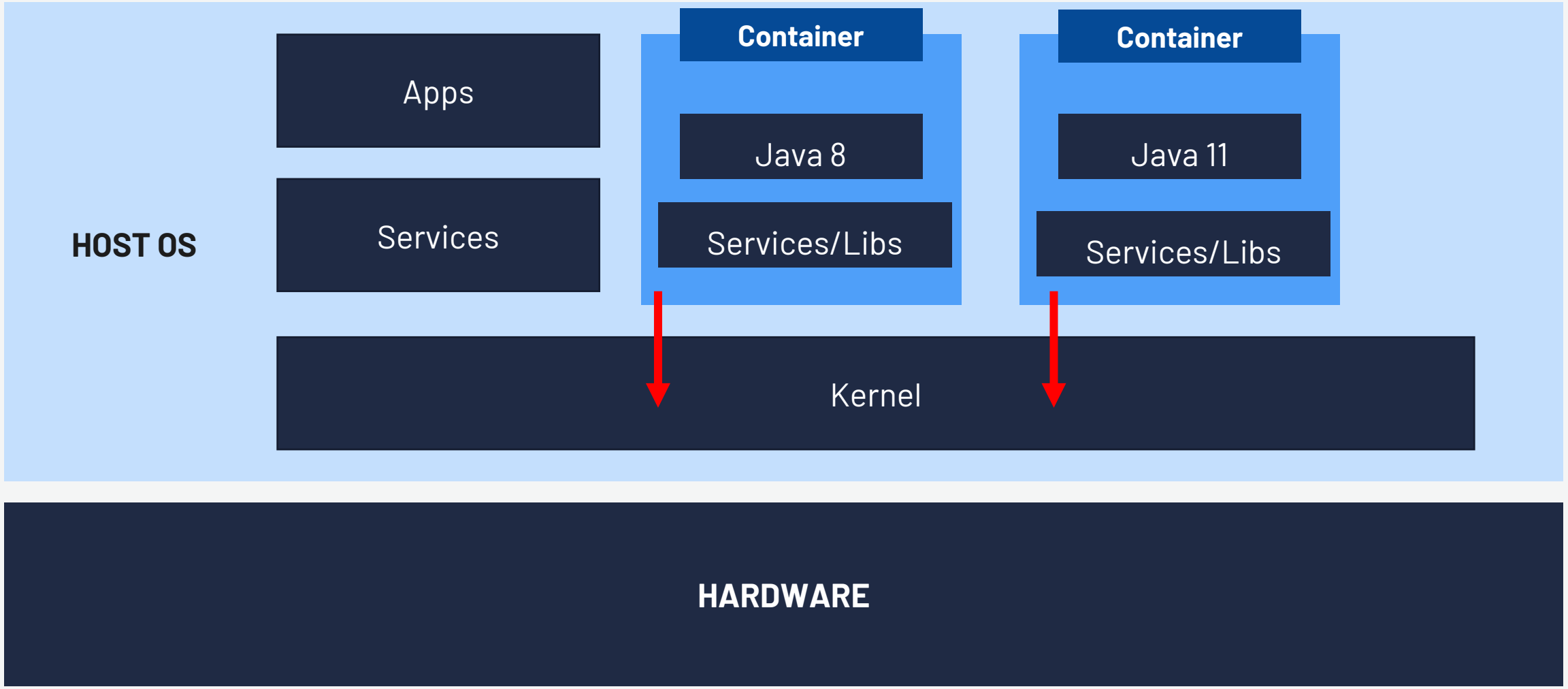
Virtual Machine

Arquitetura

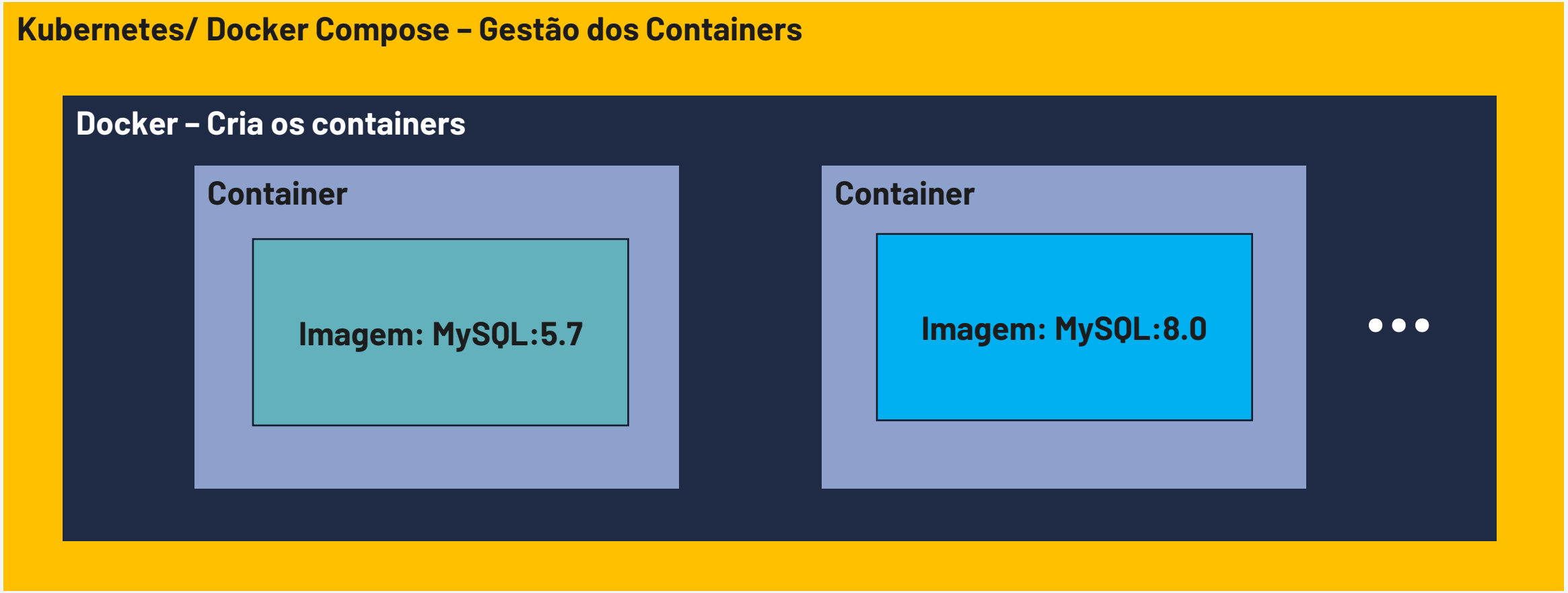


Container

Arquitetura

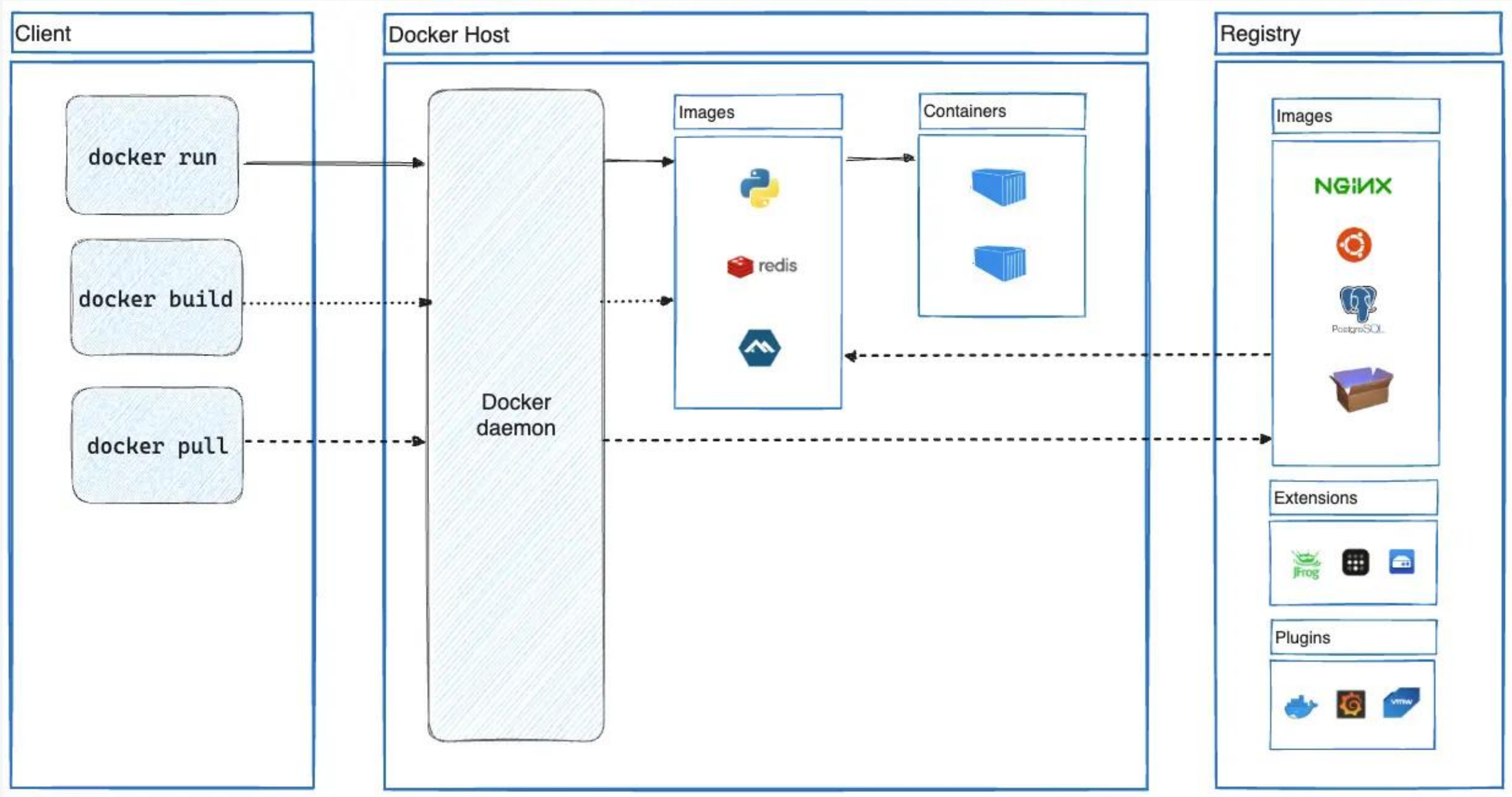


Orquestração de Container

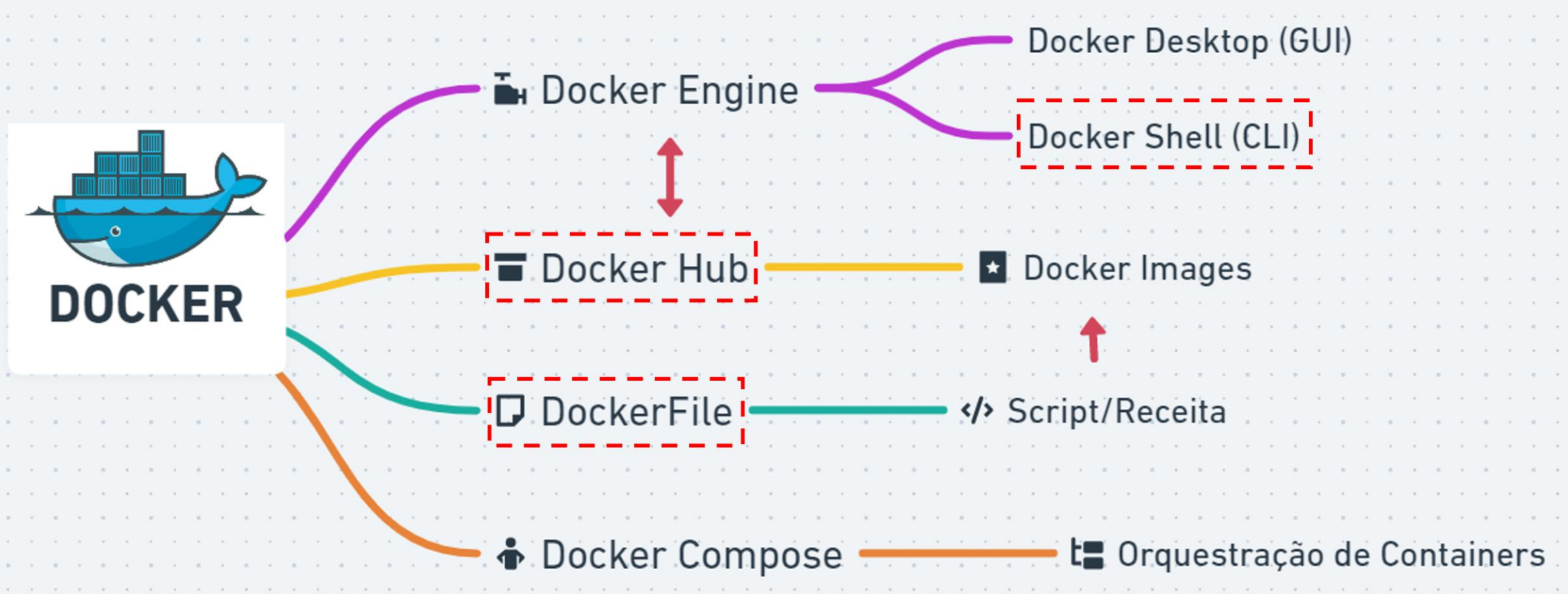


Um Container empacota sua aplicação e suas dependências (bibliotecas) de forma padronizada

Arquitetura Docker


Client
Docker Host
Registry
docker run
docker build
docker pull
Docker daemon
Images
Containers
redis
Images
NGINX
PostgreSQL
Extensions
Plugins

Ferramentas Docker



Qual é melhor?



Container vs Virtual Machine [VM]

Vários contêineres podem ser executados na mesma máquina e compartilhar o kernel do sistema operacional com outros contêineres, cada um executando como processos isolados no espaço do usuário.

Os contêineres ocupam menos espaço do que as VMs então, podem lidar com mais aplicativos, e exigem menos VMs e sistemas operacionais.

What is a Container? - Docker

Vantagens do Container

- Um container tem um tamanho muito menor que de uma VM.
- Executar um container precisa de menos recursos que uma VM.
- Provisionar um container somente toma alguns segundos enquanto que uma VM precisa esperar carregar o SO e depois a aplicação desejada.
- Containers são mais baratos por precisarem de menos recursos.
- Containers hoje são uma grande opção para micro serviços.

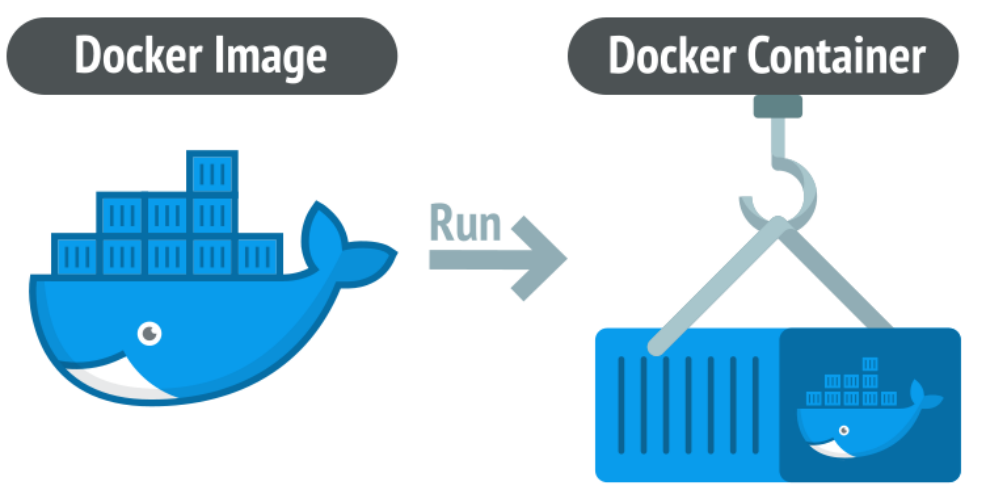
Desvantagens do Container

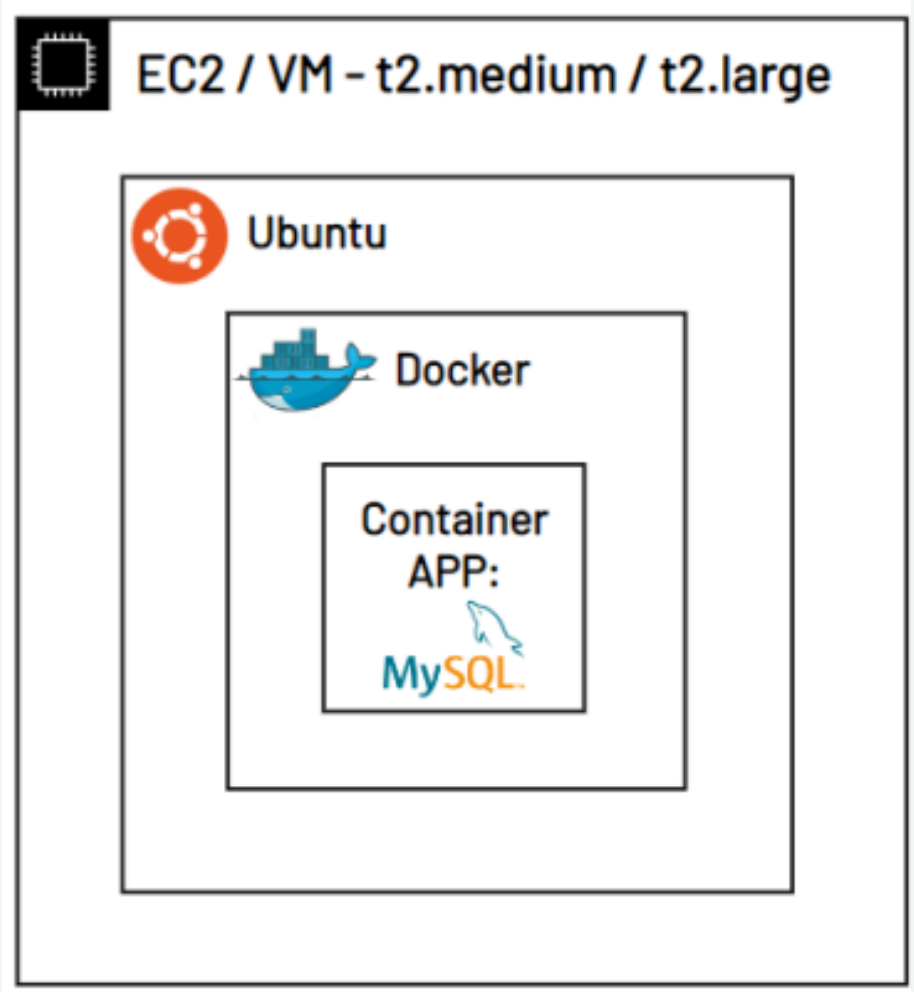
- Uma das principais desvantagens é a segurança, por compartilhar o mesmo kernel, se o kernel tem problemas o container também terá. Isso acontece por que o ambiente do container é menos apartado que uma VM.

Hands-on docker

Objetivo

Acessando remotamente uma instância Linux no serviço da EC2 pela conexão SSH, vamos instalar e configurar o Docker, com o objetivo de criar um container de MySQL.



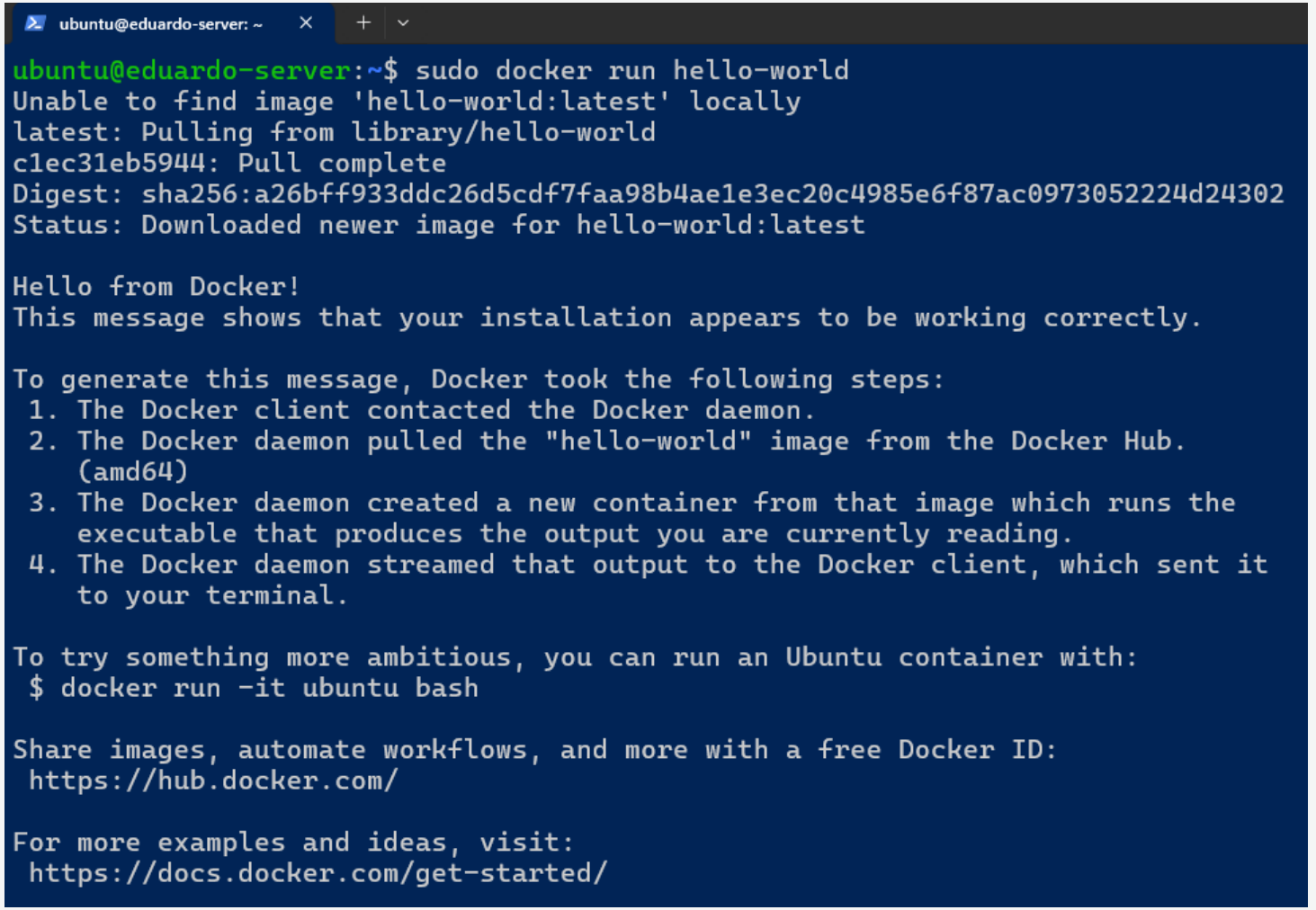


Atualizar pacotes do sistema, instalar e configurar o Docker engine

Preparando a instância

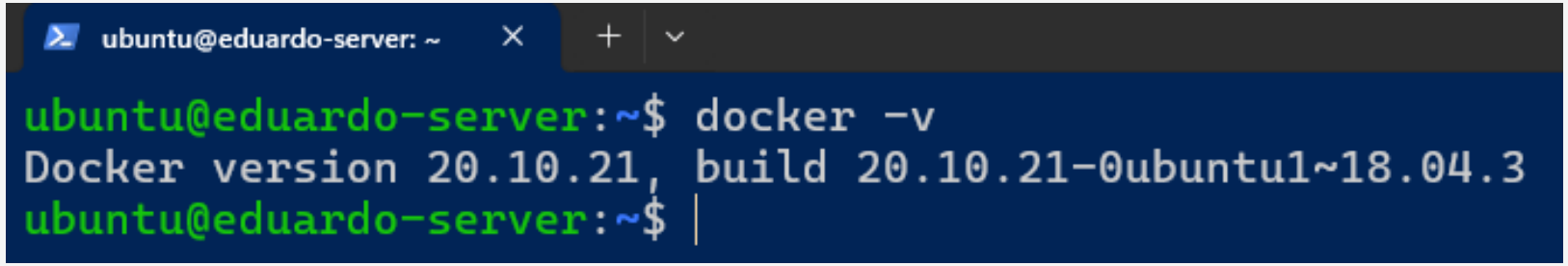
- Utlize o arquivo **`install_docker.sh`** no moodle
- Verifique se a instalação do Docker Engine foi bem-sucedida executando a imagem hello-world

`docker run hello-world`



Configurando o Docker

- Verifique a versão do Docker com: **`docker -v`**



- Ative os serviço Docker no sistema operacional para que ele fique disponível para uso: **`sudo systemctl start docker`**
- Habilite o serviço Docker para que inicie junto com o sistema operacional: **`sudo systemctl enable docker`**

Criando container de aplicação MySQL

Criando um container MySQL

- Para criar um container Docker, será necessário uma imagem da aplicação que será instalada no container para execução.
- Faça pull da imagem do MySQL que fica armazenada no repositório remoto Docker Hub. Podemos baixar a última versão, ou ainda escolher a versão que iremos utilizar: `docker pull mysql:8.0.37` por exemplo irá baixar a versão 8.0.37, se apenas utilizarmos `docker pull mysql` ele irá baixar a última versão, com a tag latest

Criando um container MySQL

- Confirme a imagem no host: `docker images`

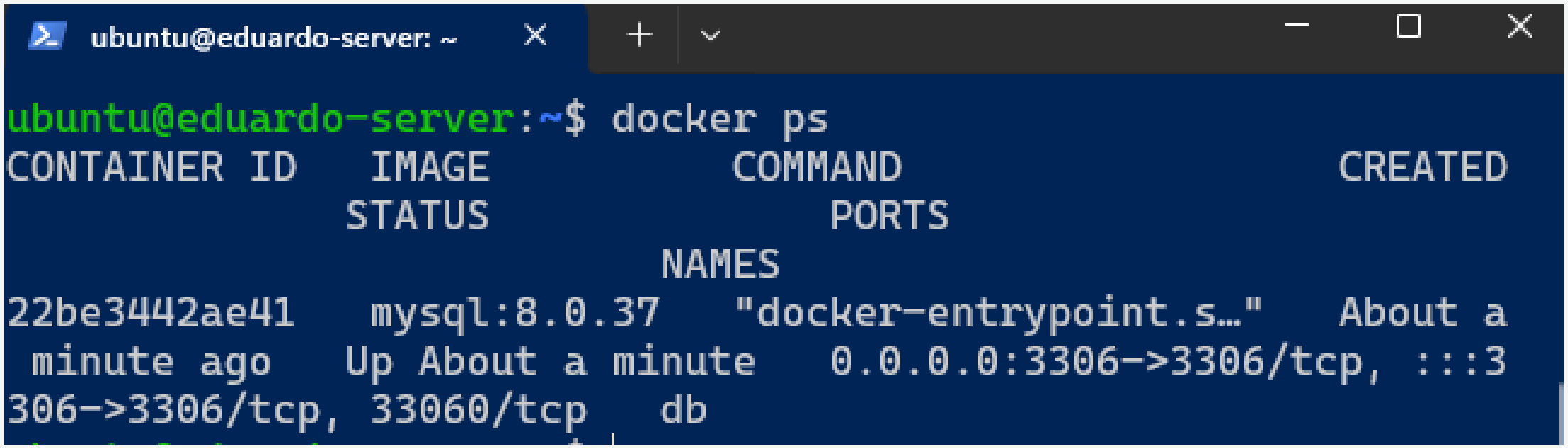
```
ubuntu@eduardo-server:~$ docker images
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
mysql               8.0.37    9a39efdaa0e6   6 days ago       574MB
eduverri/devops     2.0       411dc0c950ea   5 weeks ago      167MB
mysql               latest    82563e0cbf18   6 weeks ago      632MB
micropython         latest    f3f8ed5e63e8   3 months ago     1.12GB
python              3         86ac1e3337a9   4 months ago     1.02GB
mysql               5.7       5107333e08a8   4 months ago     501MB
hello-world         latest    d2c94e258dcb   12 months ago    13.3kB
```

Criando um container MySQL

- Agora com a imagem, vamos criar o container de MySQL. O comando abaixo **cria e inicia** um container de MySQL em segundo plano **(-d)**, com um bind de porta entre o host e o container, sendo a porta do host 3306 e do container 3306 **(3306:3306)**, com uma senha de usuário root, um nome de container e utilizando a imagem baixada
- `docker run -d -p 3306:3306 --name db -e "MYSQL_ROOT_PASSWORD=urubu100" mysql:8.0.37`

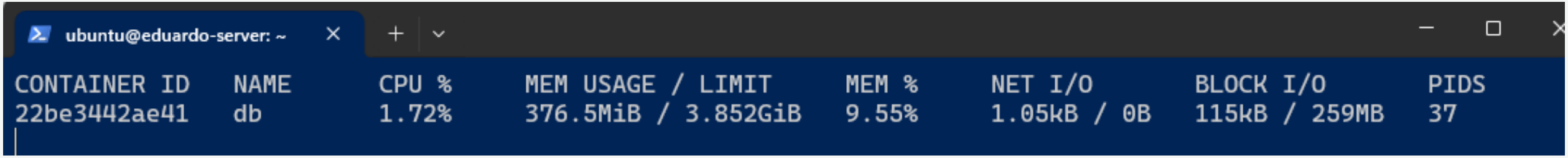
Criando um container MySQL

- Confirme se o container foi criado com `docker ps -a` ou `docker ps`



Criando um container MySQL

- Monitore o consumo de recursos de seu container com com `docker stats db`



- Para terminar o monitoramento, pressione `ctrl + c`

Acessar container de MySQL para utilizar aplicação

Acessando o terminal do container

- Para manipular a aplicação que está sendo executada dentro do container, precisamos primeiro acessar o container: **`docker exec -it db bash`**. Nesse momento, iniciamos o modo iterativo, executando um simples bash dentro do container para executar alguns comandos.

```
ubuntu@eduardo-server:~$ docker exec -it db bash
bash-5.1#
```

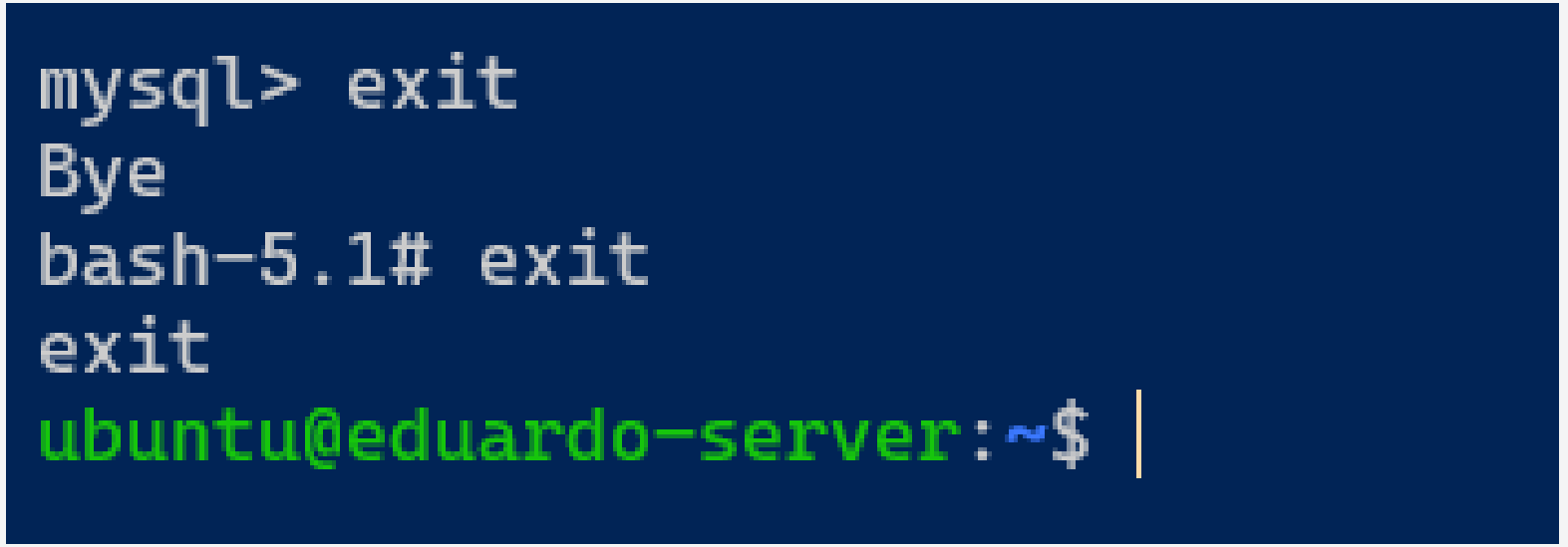
- Agora podemos iniciar o acesso ao MySQL com **`mysql -u root -p`**

```
ubuntu@eduardo-server:~$ docker exec -it db bash
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
```

Encerrando as atividades

Saindo do container

- Para fechar a conexão com MySQL, execute **`exit`**
- Continuamos no bash dentro do container, para sair e voltar ao shell-bash do Linux, execute novamente **`exit`**



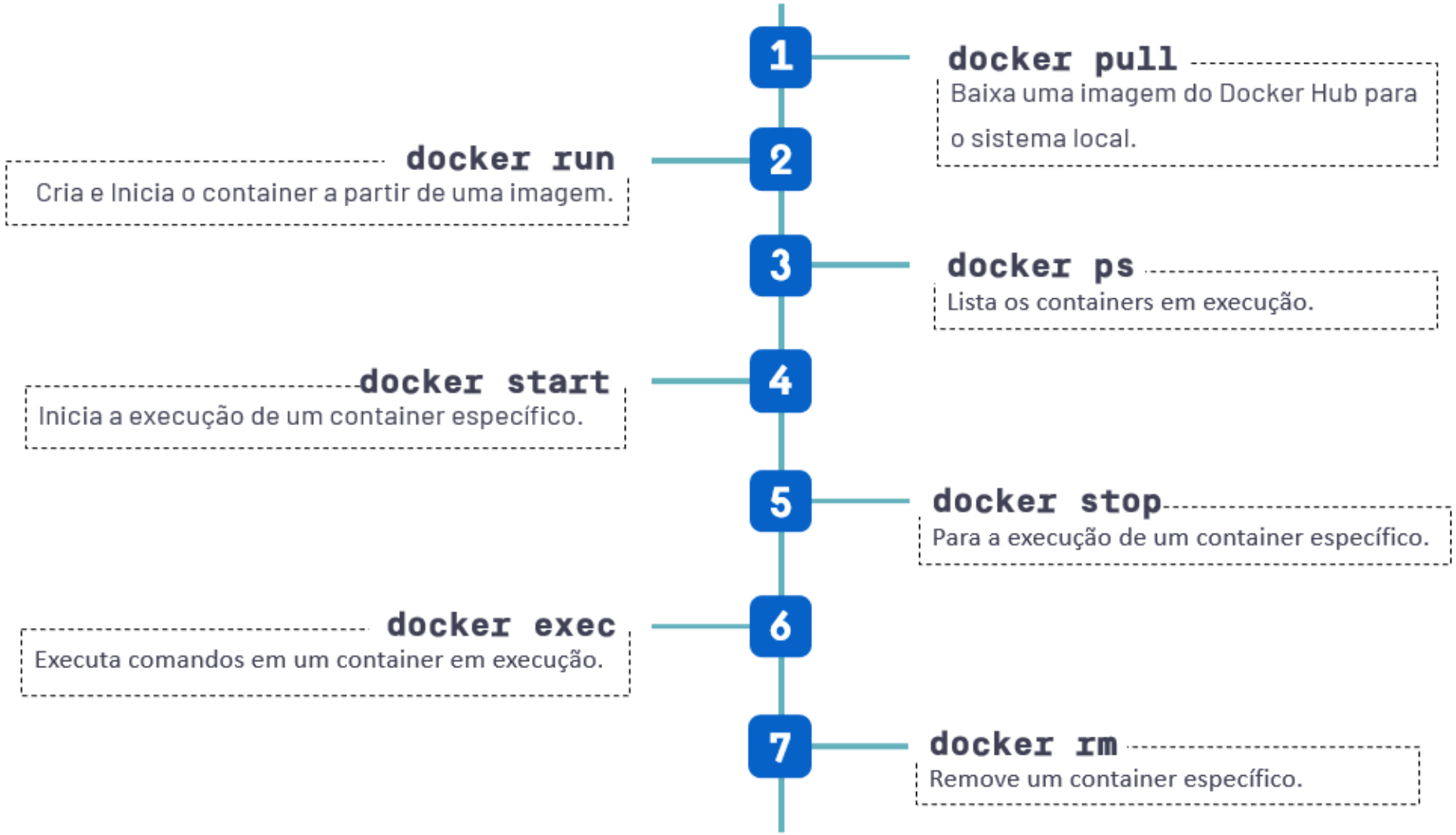
Pausando o container

- Para fechar a conexão com MySQL, execute `docker stop db`
- Para manipular o container, você pode utilizar o **ID** ou **Nome**.
- Caso precise inicar o container novamente, basta executar: `docker start db`

```
CONTAINER ID
22be3442ae41
```

```
NAMES
db
```

Comandos básicos do Docker



Agradeço
a sua atenção!

Eduardo Verri

eduardo.verri@sptech.school

SÃO PAULO TECH SCHOOL